eBPF

eBPF는 크게 네 가지 개념으로 정리할 수 있습니다.

- **Program**: 커널에서 실행될 코드 자체
- **Verifier**: 프로그램이 안전한지 로드 시점에 검사하는 검증기
- **JIT**: 검증된 바이트코드를 CPU가 직접 실행할 수 있는 기계어로 바꾸는 컴파일러
- **Hook**: 프로그램이 붙어서 실행되는 커널 내부의 특정 지점

Program

eBPF Program은 보통 **C(또는 Rust)로 작성**하고 clang/LLVM으로 컴파일해서 **eBPF 바이트코드**로 만듭니다. 일반 x86 기계어가 아니라 eBPF 전용 가상 명령어 집합입니다. 이 가상 머신은 레지스터가 11개(R0~R10)뿐이고 스택은 512바이트로 제한되는 매우 단순한 구조입니다.

프로그램은 **종류(Program Type)** 에 따라 붙을 수 있는 위치와 쓸 수 있는 기능이 정해져 있습니다. Cilium이 주로 쓰는 타입은 다음과 같습니다.

Program Type	실행 시점	Cilium에서의 용도
tc (sched_cls)	네트워크 인터페이스의 ingress/egress에서 패킷이 처리될 때	파드 veth, 물리 NIC, `cilium_host`, `cilium_vxlan`에 붙어 라우팅·정책·NAT 처리
XDP	NIC 드라이버가 패킷을 받자마자 (커널 네트워크 스택 진입 전)	NodePort/LoadBalancer 가속, DDoS 방어
cgroup/sock_addr	애플리케이션이 `connect()`, `sendmsg()` 등을 호출할 때	Service IP를 백엔드 파드 IP로 소켓 레벨에서 변환 (kube-proxy 대체)
sock_ops / sk_msg	TCP 연결 상태가 바뀌거나 소켓으로 메시지를 보낼 때	소켓 간 직접 전달 가속

프로그램이 커널의 기능을 쓰고 싶을 때는 임의의 커널 함수를 호출할 수 없습니다. 커널이 공식으로 열어둔 **Helper Function**만 호출할 수 있습니다. 자주 쓰이는 예는 다음과 같습니다.

- `bpf_map_lookup_elem()` / `bpf_map_update_elem()`: Map 조회 및 갱신
- `bpf_redirect()` / `bpf_redirect_peer()`: 패킷을 다른 인터페이스로 즉시 넘김
- `bpf_skb_store_bytes()`: 패킷의 특정 바이트 수정 (NAT 등)
- `bpf_perf_event_output()`: 사용자 공간으로 이벤트 전달 (Hubble이 쓰는 경로)

Verifier

eBPF 프로그램은 커널 안에서 실행되므로 버그가 있으면 커널 전체가 멈출 수 있습니다. 그래서 프로그램은 **로드되는 순간 Verifier의 검사**를 반드시 통과해야 합니다. Verifier는 프로그램의 모든 실행 경로를 정적으로 분석해서 다음을 확인합니다.

- **종료 보장**: 무한 루프가 없는가? (과거에는 루프 자체를 금지했고, 5.3 이후에는 횟수 상한이 증명되는 bounded loop만 허용합니다.)
- **메모리 안전**: 패킷 범위를 벗어난 읽기/쓰기가 없는가? 패킷을 읽기 전에 `data_end`와 비교하는 **경계 검사**가 코드에 있어야 합니다.
- **포인터 안전**: 초기화되지 않은 값을 쓰거나 NULL이 될 수 있는 포인터를 검사 없이 역참조하지 않는가?
- **Helper 호출 검증**: 이 프로그램 타입에서 허용되는 Helper인지, 인자 타입이 맞는지 확인합니다.
- **복잡도 제한**: 검사해야 하는 명령어 수가 한도(최신 커널은 약 100만 개)를 넘으면 거부합니다.

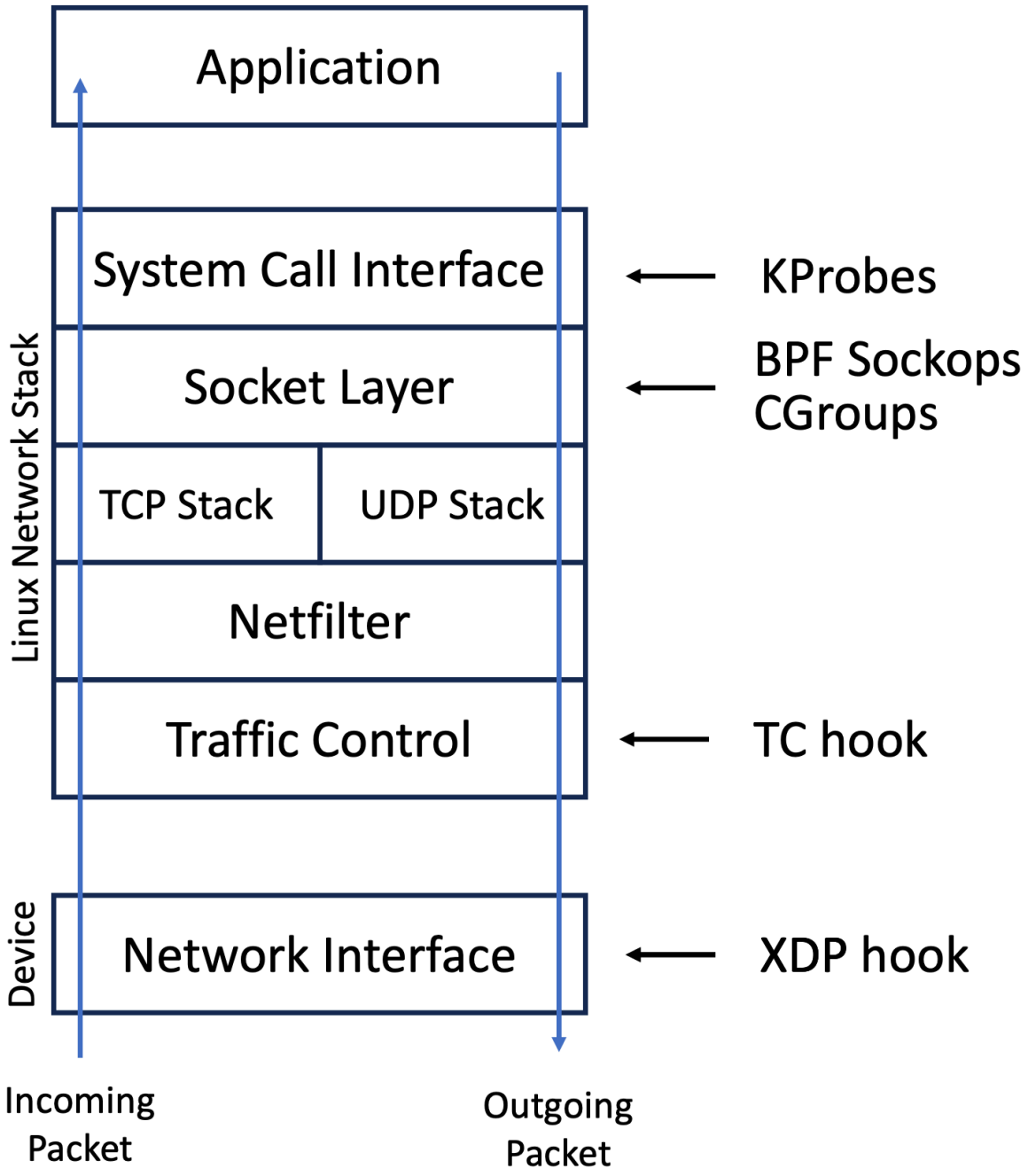
프로그램이 커지면 복잡도 한도를 넘기 쉬워서, Cilium은 하나의 큰 프로그램 대신 **Tail Call**(한 프로그램이 끝에서 다른 프로그램으로 점프하는 기능, 최대 33회)로 기능을 잘게 나눠서 연결합니다. 예를 들어 "패킷 파싱 → 정책 검사 → 전달"을 서로 다른 프로그램으로 쪼개 놓은 구조입니다.

JIT

Verifier를 통과한 바이트코드는 **JIT(Just-In-Time) 컴파일러**가 현재 CPU 아키텍처(x86_64, arm64 등)의 기계어로 변환합니다. 바이트코드를 한 줄씩 해석(interpret)하는 방식이 아니라, 로드 시점에 한 번 번역해 두고 이후에는 커널의 일반 코드와 같은 속도로 실행됩니다. 요즘 배포판은 JIT가 기본으로 켜져 있습니다.

Hook

프로그램은 Hook에 붙을 때까지는 아무 일도 하지 않습니다. Hook은 "이 이벤트가 발생하면 이 프로그램을 실행하라"는 커널 내부의 연결 지점입니다. 네트워킹에서 중요한 Hook은 다음과 같습니다.



위쪽일수록 애플리케이션에 가깝고, 아래쪽일수록 하드웨어에 가깝습니다. **XDP**는 패킷이 `sk_buff`라는 커널 자료구조로 만들어지기도 전에 실행되므로 가장 빠르고, **tc**는 `sk_buff`가 만들어진 뒤에 실행되어 더 많은 정보를 볼 수 있습니다.

eBPF Program이 커널에서 실행되는 과정

지금까지의 내용을 순서대로 연결하면 다음과 같습니다.

1. **작성 및 컴파일**: C로 짠 코드를 clang이 eBPF 바이트코드(ELF 오브젝트)로 만듭니다.
2. **로드**: 사용자 공간의 프로그램(Cilium Agent)이 `bpf()` 시스템 콜로 바이트코드를 커널에 올립니다. 이때 프로그램이 쓸 Map도 함께 생성합니다. Cilium Agent는 Go로 작성되어 있고, Go 라이브러리(`cilium/ebpf`)를 통해 이 과정을 수행합니다.
3. **검증**: Verifier가 안전성을 검사합니다. 실패하면 로드가 거부되고 에러 로그가 반환됩니다.

4. **JIT 컴파일**: 통과하면 네이티브 기계어로 변환됩니다.
5. **부착(Attach)**: 프로그램을 Hook에 연결합니다. Cilium은 파드의 veth, 노드의 `eth0` 등에 tc 프로그램을 붙이고, cgroup에 소켓용 프로그램을 붙입니다.
6. **실행**: 이벤트(패킷 도착, `connect()` 호출 등)가 발생할 때마다 커널이 프로그램을 실행합니다. 프로그램은 마지막에 **판정(verdict)** 을 반환하고, 커널은 그에 따라 동작합니다. tc 프로그램은 `TC_ACT_OK`(통과), `TC_ACT_SHOT`(드롭), `TC_ACT_REDIRECT`(다른 곳으로 전달) 등을 반환하고, XDP는 `XDP_PASS`, `XDP_DROP`, `XDP_TX` 등을 반환합니다.

한 번 붙은 프로그램은 Agent가 재시작되거나 죽어도 커널 안에서 계속 동작합니다. 그래서 Cilium Agent를 업그레이드하는 동안에도 이미 연결된 파드들의 통신은 끊기지 않습니다. 이 특성은 뒤의 BPF Map 설명과 연결됩니다.

BPF Map & Cilium Datapath

eBPF 프로그램은 스택이 512바이트뿐이고, 한 번 실행이 끝나면 상태가 사라집니다. 그런데 "이 IP의 Identity가 뭔지", "이 Service의 백엔드가 어디인지" 같은 정보는 어딘가에 계속 저장되어 있어야 합니다. 이 저장소가 **BPF Map**입니다.

BPF Map의 역할

BPF Map은 커널 메모리에 존재하는 키-값 저장소입니다. 크게 세 가지 역할을 합니다.

1. **프로그램 ↔ 프로그램 간 데이터 공유**: 같은 Map을 여러 프로그램이 참조합니다.
2. **프로그램 ↔ 사용자 공간 간 통신**: Cilium Agent가 Map에 값을 쓰면 커널의 eBPF 프로그램이 그 값을 읽어서 동작합니다. 반대로 프로그램이 통계를 쓰면 Agent가 읽어갑니다.
3. **상태 유지**: 프로그램 실행이 끝나도 Map에 담긴 데이터는 남아 있습니다. (예: 연결 추적 정보)



Agent는 "무엇을 할지"를 Map에 적어 두고, eBPF 프로그램은 패킷이 올 때마다 Map을 읽고 "어떻게 처리할지"를 결정합니다. Calico가 정책이 바뀔 때마다 iptables 규칙을 다시 쓰던 것과 달리, Cilium은 프로그램은 그대로 두고 Map의 값만 바꿉니다. 그래서 변경이 빠르고, 이미 흐르는 트래픽에 영향이 적습니다.

Map에도 용도별로 여러 종류가 있습니다.

- **Hash**: 키로 값을 찾는 일반 해시 테이블 (연결 추적, 서비스 조회)
- **LRU Hash**: 가득 차면 오래 쓰지 않은 항목부터 제거 (연결 추적 같은 임시 데이터)
- **LPM Trie**: **Longest Prefix Match**. IP 대역 단위 조회에 쓰임 (`ipcache`)
- **Array**: 인덱스로 접근하는 배열
- **Perf Event Array / Ring Buffer**: 커널에서 사용자 공간으로 이벤트를 흘려보내는 통로 (Hubble)
- **Prog Array**: Tail Call 대상 프로그램을 담는 Map

Cilium의 Map은 `/sys/fs/bpf/` 아래의 BPF 파일시스템에 고정 되어 있습니다. 앞서 Agent가 죽어도 데이터플레인이 유지된다고 한 이유가 여기에 있습니다. 프로그램과 Map이 커널에 남아 있으므로, Agent가 다시 올라와서 Map 내용을 재동기화하면 됩니다.

주요 Map: ipcache, LB, Policy

Cilium이 사용하는 핵심 Map은 다음과 같습니다.

1) `cilium_ipcache` : 이 IP가 누구??

IP 주소(또는 대역)를 **Identity**와 위치 정보로 변환하는 Map입니다. LPM Trie 타입이라 `/32` 파드 IP뿐 아니라 CIDR 단위 조회(예: 외부 `0.0.0.0/0` → `world`)도 가능합니다.

```
$ cilium-dbg bpf ipcache list
IP PREFIX/ADDRESS      IDENTITY
10.0.1.23/32           identity=1234 encryptkey=0
tunnelendpoint=0.0.0.0
10.0.2.45/32           identity=5678 encryptkey=0
tunnelendpoint=172.20.33.103
172.20.33.103/32       identity=6    encryptkey=0
tunnelendpoint=0.0.0.0     (remote-node)
0.0.0.0/0              identity=2    encryptkey=0
tunnelendpoint=0.0.0.0     (world)
```

- `10.0.1.23` 은 **로컬 파드**입니다. `tunnelendpoint` 가 `0.0.0.0` 입니다.
- `10.0.2.45` 는 **다른 노드의 파드**입니다. Identity는 5678이고 `tunnelendpoint=172.20.33.103` 이므로 이 노드로 터널링하면 됩니다.
- 1~8번대의 작은 번호는 **Reserved Identity**입니다. 자주 보이는 것은 `host` (1), `world` (2, 클러스터 밖), `remote-node` (6) 등입니다.

2) LB Map: 이 Service IP의 백엔드가 누구??

kube-proxy Replacement가 사용하는 Map들입니다. 보통 아래처럼 역할이 나뉩니다.

- **`cilium_lb4_services_*`**: `Service IP:Port` → 백엔드 목록(슬롯 번호별로 백엔드 ID 저장)
- **`cilium_lb4_backends_*`**: 백엔드 ID → 실제 `파드 IP:Port`
- **`cilium_lb4_reverse_nat`**: 응답 패킷의 출발지를 원래의 Service IP로 되돌리기 위한 역변환 정보

```
$ cilium-dbg bpf lb list
SERVICE ADDRESS      BACKEND ADDRESS (REVNAT_ID) (SLOT)
10.96.12.34:80       0.0.0.0:0 (5) (0) [ClusterIP]
                     10.0.2.45:8080 (5) (1)
                     10.0.1.88:8080 (5) (2)
```

Service 하나가 슬롯 1, 2번에 백엔드 두 개를 가지고 있는 모습입니다. 슬롯 0은 "백엔드 개수 등 메타 정보"를 담는 용도입니다. 파드가 늘거나 줄면 Agent가 이 Map의 항목만 수정하면 되고, iptables처럼 규칙 전체를 다시 쓸 필요가 없습니다.

3) Policy Map: 허용된 통신인가??

엔드포인트(파드)마다 하나씩 존재하는 Map(`cilium_policy_<엔드포인트ID>`)입니다. 키는 **(상대방 Identity, 방향, 프로토콜, 포트)** 이고, 값은 허용/차단 여부와 L7 프록시 리다이렉트 정보입니다.

```
$ cilium-dbg bpf policy get 1301
POLICY   DIRECTION   IDENTITY   PORT/PROTO   PROXY PORT   BYTES
PACKETS
Allow    Ingress     1234       8080/TCP     0            5120     40
Allow    Egress      0          ANY          0            2048     16
```

“Identity 1234(frontend)가 이 엔드포인트의 TCP 8080으로 들어오는 것을 허용한다”는 뜻입니다. 앞 글에서 설명한 Identity 기반 정책이 실제로는 이런 모양의 Map 항목입니다. Map에 **항목이 없으면 기본적으로 차단**됩니다. (정책이 적용된 엔드포인트의 경우)

eBPF Program이 Map을 참조해 Packet을 처리하는 과정

Node A의 `frontend` 파드가 `10.96.12.34:80`(Service)으로 요청하고, 이 요청이 Node B의 `backend` 파드(`10.0.2.45`)로 전달되는 과정을 따라가 보겠습니다. 터널(VXLAN) 모드 기준입니다.

단계 1. `connect()` 시점: Socket LB

```
frontend 컨테이너: connect(10.96.12.34:80)
        |
```

```
        ▼  [cgroup/sock_addr 프로그램]
        │  ① LB Services Map에서 10.96.12.34:80 조회 → 백엔드 목록
        │  ② 백엔드 선택 (10.0.2.45:8080)
        │  ③ 목적지 주소를 10.0.2.45:8080으로 바꿔서 소켓에 저장
        ▼
    이후 패킷은 처음부터 목적지가 Pod IP
```

패킷이 만들어지기도 전에 목적지가 Pod IP로 바뀌므로, **패킷 단위 DNAT와 conntrack이 필요 없습니다.** 애플리케이션은 계속 Service IP로 통신한다고 믿습니다. 응답 시 출발지 주소 복원은 `recvmsg` 시점의 소켓 훅이 Reverse NAT Map을 참조해서 처리합니다.

단계 2. 출발 노드: `from-container` (파드 veth의 tc ingress)

패킷이 파드의 veth(`lxc...`)를 통해 호스트로 들어오는 순간 tc 프로그램이 실행됩니다.

1. **패킷 파싱**: 이더넷/IP/TCP 헤더를 읽습니다. (경계 검사 필수)
2. **CT Map 조회**: 이 연결이 이미 알려진 것인지 확인합니다. 첫 SYN이므로 "신규 연결"입니다.
3. **ipcache 조회**: 목적지 `10.0.2.45`를 조회해 **Identity = 5678, tunnelendpoint = 172.20.33.103**을 얻습니다.
4. **Egress 정책 확인**: 출발지 파드에 egress 정책이 있다면, 자신의 Policy Map에서 (목적지 Identity 5678, TCP 8080)을 조회합니다.
5. **CT Map에 신규 연결 기록**
6. **전달 결정**:
   - 목적지가 **같은 노드**라면 `cilium_lxc` Map에서 대상 엔드포인트를 찾아 Tail Call로 바로 넘깁니다.
   - **다른 노드**라면 터널 키(목적지 노드 IP, **출발지 Identity**)를 설정하고 `cilium_vxlan`으로 리다이렉트합니다.

단계 3. 물리 네트워크

커널이 VXLAN으로 캡슐화해서 `eth0`을 통해 내보냅니다. VNI 필드에는 **출발지 Identity**(1234)가 실려 있습니다.

단계 4. 도착 노드: `from-overlay` → `to-container`

1. `cilium_vxlan`에 붙은 프로그램이 터널 정보에서 **출발지 Identity = 1234**를 꺼냅니다. IP를 다시 ipcache에서 조회하지 않아도 됩니다.
2. 목적지 IP로 `cilium_lxc`를 조회해 대상 엔드포인트(backend 파드)를 찾습니다.
3. Tail Call로 그 파드의 **정책 프로그램**으로 이동합니다.
4. 이 프로그램이 **backend 파드의 Policy Map**에서 키 `(Identity 1234, Ingress, TCP, 8080)`을 조회합니다.

- **항목이 있음** → CT Map에 기록하고 파드의 veth로 리다이렉트(`TC_ACT_REDIRECT`). 파드에 도착합니다.
- **항목이 없음** → 드롭하고 **Drop 이벤트**를 `cilium_events`로 보냅니다. (이 이벤트가 Hubble에서 `Policy denied DROPPED`로 보이게 됩니다.)

단계 5. 응답 패킷

응답은 CT Map에 이미 **연결 정보가 있으므로**(reply 방향 매칭) 정책을 다시 평가하지 않고 통과시킵니다. Cilium은 **Stateful**하게 동작해서, 허용된 요청의 응답을 위해 별도의 허용 규칙이 필요 없습니다.

정리

```
패킷 도착
    │
    ├─ CT Map     : 이미 아는 연결인가? (응답이면 정책 생략)
    ├─ ipcache    : 이 IP는 누구인가? 어느 노드에 있는가?
    ├─ Policy Map : 이 Identity에서 이 포트로의 접근을 허용하는가?
    ├─ LB Map     : Service IP라면 어떤 백엔드로 보낼 것인가?
    │
   ▼
판정: 통과(redirect) / 드롭 + (이벤트 → cilium_events)
```

Native Routing 모드에서는 터널 헤더가 없으므로 도착 노드가 **패킷의 출발지 IP를 ipcache에서 조회**해서 Identity를 알아냅니다. 그 외의 흐름은 같습니다.

Hubble & Troubleshooting

앞에서 모든 패킷이 eBPF 프로그램을 거치고, 프로그램이 판정을 내리는 순간(통과, 드롭, 터널 전달 등)을 알고 있다는 것을 확인했습니다. 그 정보를 사용자 공간으로 흘려보내 사람이 볼 수 있게 만든 것이 Hubble입니다.

eBPF Event → Hubble Flow

이벤트가 만들어지는 과정

1. **이벤트 발생**: eBPF 프로그램은 패킷 처리 중 의미 있는 지점에서 `bpf_perf_event_output()` Helper로 이벤트를 `cilium_events`에 씁니다. 주요 이벤트 종류는 다음과 같습니다.
   - **Trace**: 패킷이 특정 지점(파드에서 출발, 파드에 도착, 터널로 전달 등)을 지나갔다는 기록
   - **Drop**: 패킷이 드롭되었고, 그 **이유(drop reason)**
   - **Policy Verdict**: 정책 평가 결과(허용/거부)

- **Debug / Capture**: 디버깅용 (기본 비활성)

2. **Agent가 수신**: Agent의 Monitor가 이 Map을 읽습니다.
3. **디코딩과 보강(Enrichment)**: 이벤트에는 IP, 포트, Identity 숫자 같은 날것의 정보뿐입니다. Hubble은 Agent가 가진 정보(엔드포인트, Identity, K8s 메타데이터, DNS 이력)와 결합해서 **"shop/frontend-7d9 (라벨 app=frontend) → shop/backend-5c8:8080"** 같은 사람이 읽을 수 있는 형태로 바꿉니다. 이렇게 완성된 단위가 **Flow**입니다.
4. **저장**: 각 노드의 Agent는 최근 Flow를 **메모리 Ring Buffer**에 보관합니다. 용량이 고정(기본 약 4,095개)이라 오래된 Flow는 덮어씌워집니다. 즉 Hubble은 로그 저장소가 아니라 **최근 트래픽을 보는 도구**입니다. 장기 보관이 필요하면 Prometheus 메트릭이나 외부 Flow 수집을 연동해야 합니다.
5. **취합**: `Hubble Relay`가 모든 노드의 Agent에 접속해서 Flow를 모아 CLI와 UI에 제공합니다. HTTP, gRPC, Kafka 같은 L7 Flow는 Envoy 프록시가 기록한 정보로부터 만들어집니다.

Flow의 주요 필드

필드	의미
`source` / `destination`	네임스페이스, 파드명, 라벨, Identity, 워크로드 정보
`IP`, `l4`	IP 주소, TCP/UDP 포트, TCP 플래그(SYN, ACK, RST 등)
`verdict`	판정 결과 (`FORWARDED`, `DROPPED` 등, 아래 설명)
`drop_reason_desc`	드롭 사유 (`POLICY_DENIED` 등)
`traffic_direction`	`INGRESS` / `EGRESS`
`trace_observation_point`	패킷이 관측된 위치 (`to-endpoint`, `to-overlay` 등)
`is_reply`	응답 패킷 여부
`l7`	HTTP 메서드/경로/상태 코드, DNS 질의 등 (L7 가시성이 켜진 경우)
`node_name`	어느 노드에서 관측되었는가

FORWARDED / DROPPED Flow 분석

Verdict의 의미

- **FORWARDED**: Cilium 데이터플레인이 패킷을 정상적으로 다음 단계로 넘겼습니다.
- **DROPPED**: 패킷이 드롭되었습니다. `drop_reason_desc`에 이유가 있습니다.
- **AUDIT**: 정책 위반이지만 Audit 모드라서 차단은 하지 않고 기록만 남겼습니다. 정책을 운영에 적용하기 전에 영향도를 검증하는 데 유용합니다.
- **REDIRECTED**: 패킷이 L7 프록시(Envoy) 등으로 넘어갔습니다.
- **TRANSLATED**: 주소 변환(예: Service → 백엔드)이 일어났습니다.
- **ERROR**: 처리 중 오류가 발생했습니다.

실제 출력 읽기

```
$ hubble observe --namespace shop
Oct 15 10:21:03.123: shop/frontend-7d9:48222 (ID:1234) ->
shop/backend-5c8:8080 (ID:5678) to-endpoint FORWARDED (TCP Flags: SYN)
Oct 15 10:21:03.124: shop/frontend-7d9:48222 (ID:1234) <-
shop/backend-5c8:8080 (ID:5678) to-endpoint FORWARDED (TCP Flags: SYN,
ACK)
Oct 15 10:21:07.501: shop/frontend-7d9:51234 (ID:1234) <> shop/db-
0:5432 (ID:9012) policy-verdict:none INGRESS DENIED (TCP Flags: SYN)
Oct 15 10:21:07.501: shop/frontend-7d9:51234 (ID:1234) <> shop/db-
0:5432 (ID:9012) Policy denied DROPPED (TCP Flags: SYN)
```

(출력 형식과 값은 예시이며 버전에 따라 조금씩 다릅니다.)

읽는 법은 다음과 같습니다.

- **1~2행**: `frontend → backend:8080` 요청이 `to-endpoint`(목적지 파드에 전달됨) 지점에서 `FORWARDED`로 관측되었습니다. 다음 행의 SYN, ACK는 응답입니다. 연결이 정상적으로 맺어진 모습입니다.
- **3행**: `policy-verdict:none INGRESS DENIED`는 `db-0`의 **Ingress 정책 평가에서 일치하는 허용 규칙(none)이 없어서 거부**되었다는 뜻입니다.
- **4행**: 실제로 패킷이 `DROPPED`되었고 사유는 `Policy denied`입니다. 3행과 4행이 한 쌍으로 나타나는 경우가 많습니다.
- `(ID:1234)`처럼 **Identity가 함께 표시**되기 때문에, 정책의 어떤 selector가 잘못되었는지 바로 연결해서 생각할 수 있습니다.

자주 보이는 Drop 사유

사유	의미와 확인 포인트
`Policy denied`	정책에 의한 차단. Policy Map에 허용 항목이 없음
`Stale or unroutable IP`	이미 삭제된 파드의 IP 등 Cilium이 모르는 목적지
`Unknown L3 target address`	목적지 IP를 라우팅/조회할 수 없음
`CT: Map insertion failed`	연결 추적 Map이 가득 참 (Map 크기 조정 필요)
`Authentication required`	상호 인증이 필요한 정책인데 인증이 안 됨

(전체 목록이 아니며, 사유 문구는 버전에 따라 다를 수 있습니다.)

관측 지점을 보는 법

`to-endpoint`, `to-overlay`, `to-stack`, `from-endpoint` 같은 표시는 **패킷이 데이터플레인의 어느 지점에서 관측되었는지**를 나타냅니다. Node A에서 `to-overlay`까지 보이는데 Node B에서 `to-endpoint`가 안 보인다면, 문제는 두 노드 사이의 물리 네트워크(방화벽의 UDP 8472 차단, MTU 문제 등)일 가능성이 큽니다. 두 노드의 Flow를 함께 볼 수 있다는 것이 Hubble Relay의 장점입니다.

Hubble을 활용한 NetworkPolicy / 통신 문제 추적

자주 쓰는 명령어

```
# Hubble 활성화 (Cilium CLI 사용 시)
$ cilium hubble enable --ui
$ cilium hubble port-forward &       # 로컬에서 Relay 접근

# 기본 조회
$ hubble observe -f                                    # 실시간 스트리밍
$ hubble observe --namespace shop --last 100           # 최근 100개

# 필터링
$ hubble observe --verdict DROPPED                     # 드롭만
$ hubble observe --from-pod shop/frontend-7d9 --to-pod shop/backend-
5c8
$ hubble observe --to-label app=db --port 5432
$ hubble observe --type policy-verdict                 # 정책 판정 이벤트만
$ hubble observe --protocol dns --from-pod shop/frontend-7d9
$ hubble observe --to-fqdn "*.example.com"
$ hubble observe -o json                               # 모든 필드 확인
```

시나리오 1: “frontend에서 backend로 요청이 타임아웃 나요”

1단계. 드롭이 있는지 확인합니다.

```
$ hubble observe --to-pod shop/backend-5c8 --verdict DROPPED --last 20
... shop/frontend-7d9 (ID:1234) <> shop/backend-5c8:8080 (ID:5678)
Policy denied DROPPED (TCP Flags: SYN)
```

`Policy denied`가 보이면 정책 문제로 확정입니다. SYN만 있고 응답이 없다면 요청 단계에서 막힌 것입니다.

2단계. 어떤 Identity가 왔는지 확인합니다.

출발지 Identity(ID:1234)에 해당하는 라벨을 확인합니다.

```
$ cilium-dbg identity get 1234
```

기대한 라벨(`app=frontend`)이 아니거나 정책의 `fromEndpoints` selector와 맞지 않으면 원인을 찾은 것입니다. 네임스페이스 라벨이 빠졌거나(`io.kubernetes.pod.namespace`) 오타가 있는 경우가 흔합니다.

3단계. 목적지 엔드포인트의 정책 상태를 확인합니다.

```
$ cilium-dbg endpoint list                   # 엔드포인트 ID,
Ingress/Egress 정책 적용 여부
$ cilium-dbg bpf policy get <엔드포인트ID>    # Policy Map에 실제로 들어간 규
칙
```

CNP를 만들었는데 Policy Map에 항목이 없다면 selector 불일치 또는 정책 반영 실패입니다.

4단계. 정책을 수정하고 재확인합니다.

수정 후 같은 `hubble observe`로 `FORWARDED`가 보이는지 확인합니다. 3행의 `policy-verdict:...ALLOWED`가 함께 나타나는 것도 확인할 수 있습니다.

시나리오 2: 정책을 안전하게 적용하고 싶을 때 (Audit 모드)

정책을 처음 켜면 필요한 통신까지 막힐 수 있습니다. 엔드포인트에 **Policy Audit Mode**를 켜면 실제로는 차단하지 않고 `AUDIT` verdict로만 기록합니다.

```
$ cilium-dbg endpoint config <엔드포인트ID> PolicyAuditMode=Enabled
$ hubble observe --verdict AUDIT
```

`AUDIT`으로 기록된 통신을 모두 검토해서 허용 규칙을 추가한 뒤 Audit 모드를 끄면, 운영 중 장애 위험을 크게 줄일 수 있습니다.

시나리오 3: 외부 도메인으로 나가는 통신이 안 돼요 (toFQDNs)

`toFQDNs` 정책은 Cilium의 DNS 프록시가 **DNS 응답을 보고 도메인-IP 매핑을 학습**해야 동작합니다. 그래서 DNS 질의 자체가 정책에서 허용되고 프록시를 거쳐야 합니다.

```
$ hubble observe --from-pod shop/frontend-7d9 --protocol dns
... frontend -> kube-dns:53  dns-request  (query api.example.com)
FORWARDED
```

```
$ hubble observe --from-pod shop/frontend-7d9 --verdict DROPPED
... frontend -> 93.184.216.34:443 Policy denied DROPPED
```

DNS 질의는 나갔는데 이후 IP 접속이 `Policy denied`라면, 보통 **kube-dns로 가는 egress 규칙에 `rules.dns`(예: `matchPattern: "*"`)가 빠져서** Cilium이 응답을 학습하지 못한 경우입니다. DNS 요청이 아예 보이지 않으면 DNS 허용 규칙 자체가 없는 것입니다.

시나리오 4: Flow가 아예 안 보여요

`FORWARDED`도 `DROPPED`도 없다면, 패킷이 Cilium의 eBPF 프로그램에 도달하지 못한 것입니다.

- **애플리케이션이나 DNS 단계에서 실패**: Service 이름 해석 실패, 잘못된 포트 등. `--protocol dns` Flow를 먼저 확인하세요.
- **Hubble이 해당 노드에서 비활성**이거나 Relay가 그 노드에 연결되지 못함: `cilium status`, `hubble status`로 확인하세요.
- **Ring Buffer 덮어쓰기**: 트래픽이 많으면 오래된 Flow가 사라집니다. `-f`로 실시간 관측하면서 문제를 재현하세요. 이벤트가 너무 많아 유실되면 CLI가 `lost events` 경고를 보여줍니다.
- **호스트 네트워크 파드나 다른 계층의 문제**: 이 경우 노드의 `cilium-dbg monitor -t drop`으로 더 낮은 수준의 드롭 이벤트를 직접 볼 수 있습니다.

시나리오 5: FORWARDED인데 여전히 안 돼요

Flow가 `FORWARDED`이고 목적지 쪽 `to-endpoint`까지 관측되었다면 **Cilium 데이터플레인은 패킷을 파드에 전달**한 것입니다. 이 경우 문제는 보통 애플리케이션 쪽입니다(포트에서 리슨하지 않음, 앱 크래시, 앱 레벨 인증 실패 등). 응답 방향의 Flow가 아예 없다면 파드 안에서 응답을 만들지 못한 것이고, RST 플래그가 보이면 목적지가 연결을 거부한 것입니다. L7 가시성을 켜 두었다면 HTTP 상태 코드(`--http-status 500`)도 필터로 쓸 수 있습니다.

진단 순서 요약

```
 통신 문제 발생
      │
      ├─ hubble observe --verdict DROPPED  ── 드롭 있음 ─▶ drop reason 확인
      │                                                    ├─ Policy denied → Identity/라벨 → Policy Map 확인 → 정책 수정
      │                                                    └─ 그 외 사유 → ipcache, CT Map, 노드 상태 확인
      │
      ├─ 드롭 없음, FORWARDED만 있음 ──▶ to-endpoint까지 도달? → 앱/포트/응답 방향 확인
      │                                  노드 간 중간 지점에서 끊김? → 물리 네트워크(MTU, UDP 8472) 확인
```

```
      │
      └─ Flow 자체가 없음 ──▶ DNS, Hubble 상태, Ring Buffer, 호스트 네트워크 확
인
```

메트릭과 UI 활용

- **Hubble UI**의 서비스 맵에서는 드롭이 발생하는 연결이 붉게 표시되어, 문제 구간을 한눈에 찾을 수 있습니다.
- Prometheus로 `hubble_drop_total`(사유별 드롭 수) 같은 메트릭을 수집해서 **"정책 드롭이 갑자기 증가하면 알림"** 같은 운영 장치를 만들 수 있습니다.

정리

주제	핵심 개념	한 줄 요약
eBPF 동작 원리	Program, Verifier, JIT, Hook	컴파일한 프로그램을 Verifier가 검증하고, JIT로 기계어로 바꿔서, 커널 Hook에 붙여 이벤트마다 실행한다.
Hook의 종류	XDP, tc, cgroup/sock_addr	위치에 따라 처리 가능한 범위와 속도가 다르다. Cilium은 소켓(Service 변환), tc(정책/라우팅), XDP(가속)를 조합한다.
BPF Map	Hash, LRU, LPM Trie, Perf Event Array	커널 안의 키-값 저장소. Agent가 쓰고 eBPF 프로그램이 읽어서 컨트롤/데이터 플레인이 분리된다.
`ipcache`	IP → Identity, 터널 엔드포인트	"이 IP가 누구이고 어느 노드에 있는가"를 알려주는 Map
LB Map	Service → Backend → Reverse NAT	kube-proxy의 iptables 규칙을 대체하는 해시 조회
Policy Map	(Identity, 방향, 프로토콜, 포트) → 허용	엔드포인트별로 존재하며, 항목이 없으면 차단
Hubble Flow	Perf Event → Agent → Ring Buffer → Relay	eBPF가 판정하는 순간의 이벤트를 파드/라벨 정보로 보강해 사람이 읽는 Flow로 만든다.
Verdict	FORWARDED / DROPPED / AUDIT 등	DROPPED의 `drop reason`과 관측 지점(`to-endpoint` 등)이 문제 구간을 가리킨다.

정리하면, eBPF 프로그램은 Verifier의 검증과 JIT 컴파일을 거쳐 커널의 Hook에 붙고, 이벤트가 발생할 때마다 네이티브 속도로 실행됩니다. Cilium은 이 프로그램들이 참조할 정보(`ipcache`, LB, Policy, CT 등)를 BPF Map에 담아 두고, Agent가 쿠버네티스의 변경 사항을 Map에 반영하는 방식으로 컨트롤 플레인과 데이터 플레인을 분리합니다. 패킷은 Map을 조회하는 몇 번의 해시 연산만으로 "누구에게서 왔고, 어디로 가야 하며, 허용되는가"가 결정됩니다. 그리고 이 판정 지점에서 만들어지는 이벤트가 Hubble Flow가 되어, `FORWARDED` / `DROPPED`와 그 이유, 관측 지점을 통해 NetworkPolicy와 통신 문제를 추적할 수 있게 해줍니다.